Reviewer Pack — Minimal Rank of Noncrossing Partitions over Tree-like Resolu…

Entry 42aea3cf06af · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 04:32:42 UTC

Minimal Rank of Noncrossing Partitions over Tree-like Resolution Trees Complexity

Entry ID: `42aea3cf06af` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 04:32:42 UTC

1. Conjecture

Field A (mathematical branch): Noncrossing Partition Theory **Field B** (complexity object): Complexity Theory: Tseitin Resolution Trees

Statement:

> ['For every 3-CNF F with m clauses and n variables, the minimal rank of its associated noncrossing partition P(F) is at least $\Omega(m^{(1/4)n}(5/12))$.', 'This lower bound holds for all tree-like resolution proofs of F.', 'In particular, if there exists a resolution proof of F with complexity $O(m^{(1/4)n}(5/12))$, then the minimal rank of P(F) is at least $\Omega(m^{(1/4)n}(5/12))$.']

Rationale (proposer's reasoning):

> ['Noncrossing partitions have been studied in combinatorics and algebraic geometry, but their connection to complexity theory through resolution proofs is underexplored.', 'This conjecture proposes a lower bound on the minimal rank of noncrossing partitions that would imply a strong lower bound on the size of tree-like resolution proofs.', 'If true, it would provide a novel approach to proving complexity lower bounds using algebraic invariants from combinatorial geometry.']

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ea7fc440a04928b4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> To support the conjecture, the complexity of tree-like resolution proofs for generated 3-CNF instances must be at least $\Omega(m^{(1/4)n}(5/12))$ with a mean metric value not exceeding 3 across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncrossing partition theory" AND "Tseitin resolution trees" - "minimal rank" AND noncrossing AND Tseitin AND resolution AND complexity - Ω(m^(1/4)n^(5/12)) AND noncrossing partition AND resolution proof

Top relevant hits considered: - [http://arxiv.org/abs/1509.06942v2] Symmetric Decompositions and the Strong Sperner Property for Noncrossing Partition Lattices - [http://arxiv.org/abs/0706.2778v3] Chains in the noncrossing partition lattice - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_3cnf(n, m):
    clauses = []
    for _ in range(m):
        clause = set(random.sample(range(1, n+1), 3))
        if len(clause) == 3:
            clauses.append(clause)
    return clauses

def noncrossing_partition(clauses, n):
    partition = {i: [] for i in range(1, n+1)}
    for clause in clauses:
        var = random.choice(list(clause))
        partition[var].append(clause)
    return partition

def complexity(partition, n):
    max_clauses_per_var = 0
    for var in range(1, n+1):
        if var in partition and len(partition[var]) > max_clauses_per_var:
            max_clauses_per_var = len(partition[var])
    return max_clauses_per_var
```

```
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20
    m = 100
    clauses = generate_3cnf(n, m)
    partition = noncrossing_partition(clauses, n)
    metric_value = complexity(partition, n)
    instances_tested = 1
    conjecture_holds = False
    counterexample = ""

    if metric_value >= Fraction(m**(1/4) * n**(5/12), 1):
        conjecture_holds = True

    return {
        "metric_name": "complexity",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        for r in results:
            if not r['conjecture_holds']:
                counterexample = f"n={r['instances_tested']}, m={m}"
                break
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[re
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cf322fa.py", line 68, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cf322fa.py", line 51, in run_trial
    if metric_value >= Fraction(m**(1/4) * n**(5/12), 1):
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevents us from verifying whether the complexity of tree-like resolution proofs meets the conjectured lower bound. | next: Investigate and fix the error in the test code to ensure it can complete without crashing and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14456
2	preregistration	ollama_remote	glm4:latest	0	0	5683
3	novelty	ollama_remote	glm4:latest	0	0	4852
4	novelty	ollama_remote	glm4:latest	0	0	8342
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14947
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6685
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8124
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8185
9	judge	ollama_remote	glm4:latest	0	0	8541

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 79815 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/42aea3cf06af.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/42aea3cf06af.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/42aea3cf06af.tar.gz` (if generated)